Security Report

idOS

Subvisual LDA

Auditor: Miguel Palhas

Date: November 2025

Project Overview

The idOS project is preparing for a token sale. A contract will be deployed that will receive funds and allocate tokens based on a custom price discovery and allocation mechanism.

The contract is an adaptation of one that was previously developed and audited for prior projects, and was developed at https://github.com/subvisual/citizend. The new sale uses many of the same mechanics, but is also simplified in many ways.

This review focused on not only identifying potential vulnerabilities within the contract, but also to ensure that the contract’s behavior matches the requirements of this particular sale.

Scope

The audit covers all solidity code in the https://github.com/idos-network/idos-sale repository, particularly the **9664eb81619d8ba146f9617492bb92a9180962f7** commit hash, and includes:

- A review of all existing solidity code
- Writing an more comprehensive test suite
- Proposing fixes for any issues identified

Timeline

The audit was conducted over a period of one week in November, 2025.

Result

The review didn’t uncover any critical or high-severity findings.

Table of Contents

Findings Summary

Severity	Count
Critical	0
High	0
Medium	1
Low	5
Informational	1

top up for existing deposits have restrictions

ID: IDOS-5
Severity: Medium
Status: Closed

The `buy` function currently doesn’t consider whether or not the buyer is doing his first purchase or not. Each call to the function is considered as a separate purchase, and will be subject to the `minContribution`

As a consequence, with a `minContribution` of 10$, a user who previously deposited 10$, can’t later decide to increase the amount by 5$. each individual deposit must be at least 10$

Impact: Users may find it frustrating to realize they can’t necessarily top-up an existing deposit (depending on what the production parameters are)

Recommendation: The simple fix is to assert the minContribution against the total invested amount so far by that user, instead of against the new deposit

Resolution: No fix implemented since it’s not a priority, to avoid unnecessary changes to critical code paths

`rate` and `totalTokensForSale` are redundant

ID: IDOS-21
Severity: Low
Status: Closed

The contract is parameterized with:

- a rate (usdc to token, set to 0.02$)
- a total number of tokens per sale
- a min/max target (in USDC)

now that we have fixed price at the contract level, two of these values are redudant, or potentially conflicting.

assuming the minimum sale price (which is what the contract already does), the total amount of tokens for sale can be calculated trivially by the `minTarget` and `rate` values

Impact: redundant code in the smart contract. worst case scenario, mathematical errors if the values end up changing before the deploy, but only one of the two values is updated

Recommendation: remove either `rate` or `totalTokensForSale`, and derive it from the existing values

Resolution: Fixed in PR #22

token/paymentToken conversions are unnecessary

ID: IDOS-8
Severity: Low
Status: Closed

The contract has been changed so that Rising Tide now considers the USDC amount deposited, rather than the token amount requested. The implementation after this change appears to be correct, but is now doing things in a very roundabout way.

For example, it seems the following values and functions could be removed with minimal effort:

`rate`

`minPrice`

`maxPrice`

`totalTokensForSale` (replaced by `maxTarget`)

`tokenToPaymentToken`

`paymentTokenToToken`

The main caveat to this is regarding the `buy` function, which currently tokens an amount denominated in tokens, not payment tokens. This means that fully removing these outdated calculations results in a slight public API change. It’s not clear whether this is doable or desirable to do at this stage on the corresponding UI

Impact: A lot of clutter currently exists in the contract that hinders readability, clarity, and the ability to write tests.

Recommendation: Remove everything that is safe to remove without causing breaking changes to public API. Analyze if the additional breaking changes are also doable

Resolution: Fixed in PR #12

minPrice and maxPrice are the same hardcoded values

ID: IDOS-3
Severity: Low
Status: Closed

The Sale contract originally had a dynamic price ranging between `minPrice` and `maxPrice`.

In recent changes that feature was apparently scrapped, and the two values were made equal

In practice, the two values are now redundant and some now useless computations are being done with the assumption that they’re different.

https://github.com/subvisual/idos-sale/blob/9e77c3e1fe8cbc47c05fe691252cb31c08132027/src/Sale.sol#L157-L158

https://github.com/subvisual/idos-sale/blob/9e77c3e1fe8cbc47c05fe691252cb31c08132027/src/Sale.sol#L293-L304

Impact: Make it unnecessarily hard (for auditors and investors) to understand the rules of the contract

Recommendation: replace these two hardcoded values with a single value provided during contract deploy.

Resolution: Fixed in PR #9

Minor: unused state variable

ID: IDOS-2
Severity: Low
Status: Closed

The contract includes a `token` state variable, described in comments as intending to point to an ERC20 token. this seems to be a leftover from a previous version of the contract. This token is set by an admin-only function, but is never actually used in the code.

https://github.com/subvisual/idos-sale/blob/9e77c3e1fe8cbc47c05fe691252cb31c08132027/src/Sale.sol#L337-L340

https://github.com/subvisual/idos-sale/blob/9e77c3e1fe8cbc47c05fe691252cb31c08132027/src/Sale.sol#L56-L57

Impact: Potentially makes it confusing or suspicious to investors that will look into the code and find an invalid `token`.

Recommendation: Remove the variable and setter function

Resolution: Fixed in PR #13

Minor: startRegistration & endRegistration are dead code

ID: IDOS-1
Severity: Low
Status: Closed

The sale contract tracks two values `startRegistration` and `endRegistration`, provided at the constructor, and updateable via admin calls

These two values seem to have no use whatsoever on the contract.

https://github.com/subvisual/idos-sale/blob/9e77c3e1fe8cbc47c05fe691252cb31c08132027/src/Sale.sol#L136-L137

https://github.com/subvisual/idos-sale/blob/9e77c3e1fe8cbc47c05fe691252cb31c08132027/src/Sale.sol#L346-L352

Recommendation: remove these values from the constructor, and the two corresponding setter functions

Impact: None in production, just clear contract behavior for everyone

Resolution: Fixed in PR #20

The contract has a maximum number of investors

ID: IDOS-4
Severity: Informational
Status: Closed

The contract currently refuses further investment once `investorCount >= maxTarget / minContribution`, where:

- investorCount is how many individual investors have deposited
- maxTarget is maximum USDC amount we want to raise
- minContribution is the smallest possible USDC contribution we accept from each user

this makes it so that, assuming a `maxTarget` of 100$, and a `minContribution` of 10$, we effectively close the sale after 10 individual investors.

From discussing with the team, it seems the goal here is to instead let the sale continue, and allow the rising tide mechanism to calculate a cap smaller than the minimum contribution. Meaning in the example above, we would accept a total of 20 investors, where each one would be capped at 5$ (less than the original minContribution)

https://github.com/subvisual/idos-sale/blob/10c757ffe618999831f42bb40c03eb4a9b27ba1f/src/Sale.sol#L217-L219

Impact: The total amount would still be raised, but in the event of a fully successful sale, later users would be locked out, instead of everyone ending up with a smaller overall cap.

Recommendation: confirm this is the intended behaviour, and if so, remove this check from the contract's `buy` function

Resolution: Fixed in PR #11